

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track · Windows · Terminal-first

MONTH 6 — LAB GUIDE

Build a Prediction Agent

An agent that learns from data and predicts outcomes — a model you train by hand.

The capstone of the from-scratch era. You will finally see, with total clarity, how a model actually learns.

What you will build this month

- An agent that learns the relationship in data and predicts new values from it.
- It starts with a wild guess and improves itself, step by step, until it fits the data.
- You will code the learning process — gradient descent — entirely by hand.
- It is tested honestly on unseen data and shipped to GitHub as v1.0.

Why this is the most important month so far

- Last month your agent learned by counting. This month it learns by **gradually improving a guess** — the method behind almost all modern AI, including neural networks and LLMs.
- You will build **gradient descent** yourself, line by line. Once you have seen it work with your own eyes, AI stops being magic.
- Go slowly and enjoy it. This is the concept that ties the whole from-scratch era together.

Month 6 at a glance

This is the capstone of everything you have built by hand. A prediction agent learns a pattern from data and uses it to predict new outcomes — like guessing a house price from its size, or a score from hours studied. The heart of it is a process called gradient descent: start with a rough guess, measure how wrong it is, and nudge it a little less wrong, over and over, until it fits. You will build every piece of that yourself.

Week	Focus	What you ship
Week 1	Prediction & the idea of a line	A program that predicts with a guessed line (v0.1)
Week 2	Measuring error (the loss)	A program that scores how wrong its line is (v0.2)
Week 3	Gradient descent: learning by hand	A model that trains itself to fit the data (v0.3)
Week 4	Train/test, real data & ship	A tested prediction agent on real data (v1.0)

The journey this month: Week 1 you predict with a line you guess. Week 2 you measure exactly how wrong the guess is. Week 3 — the big one — the model improves itself automatically with gradient descent. Week 4 you test it fairly on real data.

Week 1 — Prediction & the idea of a line

Goal of the week: understand prediction as drawing a line through data, and make predictions using a line you choose by hand.

Day 1 — What is prediction?

Prediction means using a known pattern to guess an unknown value. If more study hours usually mean higher scores, we can predict a score from hours.

Task 1.1 — Set up and see the data

1. Create a folder `predictor`, open it in VS Code, and make `day1.py`:

```
# hours studied -> exam score
hours = [1, 2, 3, 4, 5]
scores = [52, 58, 62, 71, 78]

for h, s in zip(hours, scores):
    print(h, "hours ->", s, "score")
```

2. Save and run. You can see the pattern: more hours, higher score.

✓ What you should see

```
1 hours -> 52 score
2 hours -> 58 score
3 hours -> 62 score
4 hours -> 71 score
5 hours -> 78 score
```

`zip(hours, scores)` pairs the two lists together so you can loop over them side by side. Our goal: find a rule that turns any “hours” into a predicted “score”.

 **Checkpoint:** You understand prediction as finding a pattern in data to guess new values.

Day 2 — A line is just: $y = m \cdot x + c$

A straight line predicts an output (y) from an input (x) using two numbers: the slope m (how steep) and the intercept c (where it starts). That's the whole model.

Task 2.1 — Predict with a guessed line

3. New file `day2.py`:

```
m = 6          # slope: score goes up ~6 per hour (a guess)
c = 46         # intercept: score at 0 hours (a guess)

def predict(x):
    return m * x + c

print("Predict 3 hours:", predict(3))
print("Predict 5 hours:", predict(5))
```

✓ What you should see

Predict 3 hours: 64
Predict 5 hours: 76

Every prediction model, no matter how advanced, is really just “take the input, do some math with tunable numbers, get an output.” Here the tunable numbers are **m** and **c**.

🚩 **Checkpoint:** You can make predictions with a simple line defined by a slope and intercept.

Day 3 — Plot the line against the data

To see how good your guessed line is, draw it over the real data points (using Matplotlib from Month 3).

Task 3.1 — Visualise the fit

4. New file **day3.py** (install matplotlib if needed with `pip install matplotlib`):

```
import matplotlib.pyplot as plt
hours = [1, 2, 3, 4, 5]
scores = [52, 58, 62, 71, 78]
m, c = 6, 46

plt.scatter(hours, scores)           # real data as dots
line_y = [m*x + c for x in hours]
plt.plot(hours, line_y, color="red") # your line
plt.xlabel("Hours"); plt.ylabel("Score")
plt.show()
```

5. Save and run. A chart shows the real dots and your red prediction line through them.

✓ What you should see

A chart window: 5 dots with a red line running near them

🚩 **Checkpoint:** You can see your prediction line drawn against the real data.

Days 4–7 — Adjust by hand & ship v0.1

- Day 4: Change **m** and **c** by hand and re-plot. Try to make the red line pass as close to the dots as you can. Notice this is hard to do by eye!
- Day 5: Realise the core problem — we need a way to MEASURE how good a line is, not just eyeball it. (That's next week.)
- Day 6: Review errors and make a clean `predict(x, m, c)` function that takes the line's numbers as inputs.
- Day 7: Build v0.1 of `predictor.py`: the data, a predict function, and a plot of a hand-chosen line. Run it start to finish.

🚩 **Checkpoint:** 🏁 Week 1 done. You can predict with a line and see the fit. Next week: measuring how wrong it is.

Week 2 — Measuring error (the loss)

Goal of the week: you can't improve what you can't measure. This week you compute exactly how wrong a line is — a single number called the loss.

Day 8 — The error of one prediction

For one data point, the error is simply: how far the prediction is from the real value.

Task 8.1 — Error for a single point

6. New file `day8.py`:

```
def predict(x, m, c):  
    return m * x + c  
  
real = 62  
guess = predict(3, 6, 46)    # predict 3 hours  
error = guess - real  
print("Guess:", guess, "Real:", real, "Error:", error)
```

✓ What you should see

Guess: 64 Real: 62 Error: 2

Error = prediction – reality. A positive error means we guessed too high; negative means too low. We want errors as close to zero as possible.

🚩 **Checkpoint:** You can measure how far a single prediction is from the truth.

Day 9 — Why we square the errors

Some errors are positive, some negative — if we just added them they'd cancel out and hide how wrong we are. Squaring makes every error positive and punishes big mistakes more.

Task 9.1 — Squared error

7. Add:

```
errors = [2, -3, 5, -1]  
print("Just added (misleading):", sum(errors))  
squared = [e*e for e in errors]  
print("Squared (honest):", squared)  
print("Sum of squared:", sum(squared))
```

✓ What you should see

Just added (misleading): 3

Squared (honest): [4, 9, 25, 1]

Sum of squared: 39

Adding raw errors gives a tiny 3 — hiding that we were quite wrong. Squaring turns them all positive and reveals the true size of the mistakes.

🚩 **Checkpoint:** You understand why we square errors — to stop them cancelling and to punish big misses.

Day 10 — The loss: one number for the whole line

The loss (called Mean Squared Error) is the average of all the squared errors. One number that says how wrong the WHOLE line is. Lower = better.

Task 10.1 — Compute the loss

8. New file `day10.py`:

```
hours = [1, 2, 3, 4, 5]
scores = [52, 58, 62, 71, 78]

def loss(m, c):
    total = 0
    for x, real in zip(hours, scores):
        guess = m * x + c
        total = total + (guess - real) ** 2
    return total / len(hours)

print("Loss for m=6, c=46:", loss(6, 46))
print("Loss for m=2, c=50:", loss(2, 50))
```

✓ What you should see

Loss for m=6, c=46: 3.6

Loss for m=2, c=50: 106.0

The first line (m=6) has a much smaller loss than the second (m=2), which means it fits the data far better. Now “good line” has a precise meaning: low loss.

 **Checkpoint:** You can score any line with a single loss number. Lower loss = better fit.

Days 11-14 — Explore the loss & ship v0.2

- Day 11: Try many values of `m` by hand and print the loss for each. Watch the loss fall then rise — there's a “best” `m` in the middle.
- Day 12: Plot loss against different `m` values. You'll see a U-shaped curve (a valley). The bottom of the valley is the best line.
- Day 13: Realise the big idea — training a model just means finding the bottom of that valley automatically. (That's next week!)
- Day 14: Ship v0.2 — a program that computes and plots the loss for different lines.

 **Checkpoint:** End of Week 2: you can measure exactly how good any line is. You've found the valley — next you'll roll down it.

Week 3 — Gradient descent: learning by hand

Goal of the week: the heart of the whole course. Instead of guessing lines, your model will improve itself automatically by rolling downhill to the lowest loss — gradient descent.

Day 15 — The idea: roll downhill

Imagine the loss curve as a valley and your current line as a ball on its slope. Gradient descent = repeatedly step the ball a little way downhill until it settles at the bottom (lowest loss).

The recipe (memorise this rhythm): 1) see which way is downhill, 2) take a small step that way, 3) repeat. That's gradient descent — and it's how neural networks and LLMs learn too.

 **Checkpoint:** You understand gradient descent as repeatedly stepping downhill to the lowest loss.

Day 16 — Which way is downhill? (the gradient)

The gradient tells us which way the loss increases; we step the opposite way. For a line, there's a known formula for the slope of the loss with respect to m and c .

Task 16.1 — Compute the gradients

9. New file `day16.py` — this function returns how much to nudge m and c :

```
def gradients(m, c):
    grad_m = 0
    grad_c = 0
    n = len(hours)
    for x, real in zip(hours, scores):
        error = (m * x + c) - real
        grad_m = grad_m + 2 * error * x
        grad_c = grad_c + 2 * error
    return grad_m / n, grad_c / n
```

Don't worry about memorising the formula. The key idea: it uses the errors to work out which direction reduces the loss for m and c . A positive gradient means “decrease this number”; negative means “increase it”.

 **Checkpoint:** You have a function that says which way to nudge m and c to reduce the loss.

Day 17 — Take one step

A step = move m and c a little in the downhill direction. How big a step is the learning rate.

Task 17.1 — One update step

10. Add:

```
m, c = 0, 0          # start with a wild guess
learning_rate = 0.01

gm, gc = gradients(m, c)
m = m - learning_rate * gm
c = c - learning_rate * gc
```

```
print("After one step: m =", round(m,3), "c =", round(c,3))
```

`m = m - learning_rate * gm` nudges m downhill. The minus sign is what makes it go DOWN the loss. The learning rate keeps the step small and safe.

🚩 **Checkpoint:** You can take a single downhill step that improves m and c.

Day 18 — The training loop

Now repeat that step hundreds of times and watch the loss fall. This IS training a model.

Task 18.1 — Train!

- Put it together in `predictor.py`:

```
m, c = 0, 0
learning_rate = 0.01

for step in range(1000):
    gm, gc = gradients(m, c)
    m = m - learning_rate * gm
    c = c - learning_rate * gc
    if step % 200 == 0:
        print("step", step, "loss", round(loss(m, c), 2))

print("Learned line: m =", round(m,2), "c =", round(c,2))
```

✓ What you should see

```
step 0 loss 4100.0 ...
step 200 loss 9.7
step 400 loss 4.1
step 800 loss 3.6
Learned line: m = 6.4 c = 45.6
```

If the loss grows to huge numbers or shows “nan”, your learning rate is too big. Lower it (try 0.001). Too big a step overshoots the valley.

🚩 **Checkpoint:** 🎉 Your model TRAINED ITSELF — the loss fell from thousands to near zero, and it discovered the line on its own. This is machine learning.

Days 19-21 — Watch it learn & ship v0.3

- Day 19: Plot the loss at every step — you'll see the classic falling curve of a model learning.
- Day 20: Animate or re-plot the line every 100 steps to watch it swing into place over the data. Deeply satisfying.
- Day 21: Commit and push to GitHub. This is v0.3.

🚩 **Checkpoint:** You built and trained a model from scratch with gradient descent, and watched it learn.

Week 4 — Train/test, real data & ship v1.0

Goal of the week: make it a trustworthy agent — test it fairly on unseen data, run it on a real dataset, compare to a professional library, and ship v1.0.

Day 22 — Split into train and test

The golden rule again (from Month 5): train on some data, test on data the model never saw, so you know it really learned the pattern rather than memorising.

Task 22.1 — Hold back test points

12. Split your data:

```
train_hours = [1, 2, 3, 4]
train_scores = [52, 58, 62, 71]
test_hours = [5] # kept hidden during training
test_scores = [78]
```

13. Train using only the train data, then predict the test point and compare to its real value.

If it predicts the unseen point well, it truly learned the pattern. This is how you trust a model.

🚩 **Checkpoint:** You can train on one part of the data and test honestly on another.

Day 23 — Predict new, unseen inputs

Task 23.1 — Make it an agent

14. Wrap the trained model in a simple loop that predicts any value the user types:

```
while True:
    text = input("Enter hours (or quit): ")
    if text == "quit":
        break
    x = float(text)
    print("Predicted score:", round(m * x + c, 1))
```

15. Run it and ask for predictions on values not in your data, like 6 or 2.5.

`float(text)` turns the typed text into a decimal number so you can do math with it.

🚩 **Checkpoint:** Your trained model is now an interactive agent that predicts any new input.

Day 24 — Compare to a professional library

- Use scikit-learn's `LinearRegression` on the same data (`pip install scikit-learn`).
- Confirm its line is almost identical to the one YOUR hand-built gradient descent found. Proof you built the real thing.

Day 25 — Real data & robustness

- Load a real CSV (Month 3 skills) — e.g. house size vs price — and train your predictor on it.
- Handle bad input (non-numbers, empty) gracefully so the agent never crashes.

Days 26–30 — Final polish & ship v1.0

- Day 26: Test on the real dataset. Report the test loss honestly. Tune the learning rate and steps if needed.
- Day 27: Write a **README.md** explaining what it predicts and how gradient descent trains it, in your own words.
- Day 28: Record a short demo — show the loss falling during training, then live predictions.
- Day 29: Final commit and push (`git add .` → `git commit -m "v1.0"` → `git push`).
- Day 30: Show it off. You built a model that trains itself — the beating heart of modern AI.

Month 6 complete — and the from-scratch era with it

This was the milestone the whole first half was building toward. You now understand, with total clarity, how a model learns: make a guess, measure the loss, follow the gradient downhill, repeat. That single loop — gradient descent — is exactly how neural networks and even large language models are trained, just at enormous scale. AI is no longer magic to you; it's a process you have built with your own hands. Next comes the exciting pivot: you'll connect to a real large language model and start building genuinely intelligent, agentic AI — standing on the deep foundation you've earned over these six projects.